

Scrabble IST 2

Neste segundo projeto de Fundamentos da Programação será implementada uma versão mais completa do jogo de Scrabble, cuja versão básica foi implementada no primeiro projeto. Para este efeito, deverá definir um conjunto de tipos abstratos de dados que deverão ser utilizados para manipular a informação necessária no decorrer do jogo, através da refactorização de código já desenvolvido no primeiro projeto ou do desenvolvimento de código novo. Será necessário ainda implementar um conjunto de funções adicionais com novas funcionalidades.

1 Descrição do jogo

As regras do jogo de Scrabble a considerar são as mesmas das descritas no enunciado do primeiro projeto¹. No entanto, nesta versão do jogo é definido um **vocabulário** de palavras aceites, que será utilizado, não só para verificar se as jogadas de cada jogador são válidas, mas também para automatizar o jogo, permitindo que um ou mais jogadores sejam o próprio computador. Adicionalmente, também para permitir a automatização do jogo, será preciso gerar todos os **subpadrões** válidos dado um **padrão** formado pelas letras e espaços entre duas casas do tabuleiro.

1.1 Vocabulário e procura

O vocabulário do jogo é formado por uma ou mais palavras, isto é, por cadeias de caracteres de 2 a 15 letras maiúsculas do alfabeto português. A construção do vocabulário deve permitir a procura acelerada de palavras pelo seu comprimento e, de entre as palavras com o mesmo comprimento, pela sua primeira letra.

1.2 Padrões e subpadrões

Um padrão é uma sequência de letras e espaços livres entre duas casas do tabuleiro, numa linha horizontal ou vertical. Para permitir ao computador jogar, será preciso gerar todos os padrões possíveis. Para este fim, considere o seguinte algoritmo. Dado um padrão p , o conjunto de todos os sub-padrões possíveis são todas as cadeias de caracteres $p[i : j]$, em que $0 \leq i < \text{len}(p)$ e $i < j \leq \text{len}(p)$. Por exemplo, para o padrão $p = \text{' .R.LA.A. '}$:

¹<https://fenix.tecnico.ulisboa.pt/downloadFile/2252418289016459/FP-2025-P1.pdf>

	i=0	i=1	i=2	i=3	i=4	i=5	i=6	i=7
j=8	.R.LA.A.	R.LA.A.	.LA.A.	LA.A.	A.A.	.A.	A.	.
j=7	.R.LA.A	R.LA.A	.LA.A	LA.A	A.A	.A	A	
j=6	.R.LA.	R.LA.	.LA.	LA.	A.	.		
j=5	.R.LA	R.LA	.LA	LA	A			
j=4	.R.L	R.L	.L	L				
j=3	.R.	R.	.					
j=2	.R	R						
j=1	.							

Note que o padrão *original* p é por sua vez um sub-padrão de si próprio, correspondente a não retirar qualquer caráter nem do início nem do fim do padrão *original*. Note também que alguns dos sub-padrões gerados podem ser inviáveis, porque não correspondem a jogadas válidas de acordo as regras do jogo. Assim, são inviáveis todos os sub-padrões:

- (1) que não contenham qualquer letra e sejam apenas constituídos por espaços livres, porque todas a jogadas válidas devem usar pelo menos uma letra do tabuleiro;
- (2) que não contenham qualquer espaço livre e sejam apenas constituídos por letras, porque todas as jogadas válidas devem usar pelo menos uma das letras do jogador;
- (3) para os quais exista no padrão *original* uma letra na posição anterior à da primeira posição do sub-padrão, ou exista no padrão *original* uma letra na posição seguinte à última posição do sub-padrão, para não sobrepor ou colar com palavras já existentes no tabuleiro².

Riscando os sub-padrões inviáveis do exemplo anterior, obtemos:

	i=0	i=1	i=2	i=3	i=4	i=5	i=6	i=7
j=8	.R.LA.A.	R.LA.A.	.LA.A.	LA.A.	A.A.	.A.	A.	.
j=7	.R.LA.A	R.LA.A	.LA.A	LA.A	A.A	.A	A	
j=6	.R.LA.	R.LA.	.LA.	LA.	A.	.		
j=5	.R.LA	R.LA	.LA	LA	A			
j=4	.R.L	R.L	.L	L				
j=3	.R.	R.	.					
j=2	.R	R						
j=1	.							

Assim, todos os sub-padrões válidos ordenados (de cima para baixo e a seguir de esquerda para a direita) do exemplo anterior são ('.R.LA.A.', '.R.LA.A', '.R.LA', '.R', 'R.LA.A.', 'R.LA.A', 'R.LA', 'LA.A.', 'LA.A', 'A.').

Para gerar todos os padrões possíveis duma instância tabuleiro, será preciso usar o algoritmo acima a partir do padrão *original* obtido para cada linha e coluna completa desse tabuleiro.

²Esta versão do Scrabble ignora restrições nas linhas adjacentes quando jogamos na horizontal e nas colunas adjacentes quando jogamos na vertical.

2 Trabalho a realizar

Um dos objetivos deste segundo projeto é definir e implementar um conjunto de Tipos Abstratos de Dados (TAD) que deverão ser utilizados para representar a informação necessária, bem como um conjunto de funções adicionais que permitirão executar corretamente o jogo Scrabble.

2.1 Tipos Abstratos de Dados

Atenção:

- Apenas os construtores e as funções para as quais a verificação da correção dos argumentos é explicitamente pedida devem verificar a validade dos argumentos.
- Os modificadores, e as funções de alto nível que os utilizam, alteram de modo destrutivo o seu argumento.
- As barreiras de abstração devem ser sempre respeitadas.

2.1.1 TAD *casa* (1,0 valores)

O TAD imutável³ *casa* é usado para representar uma casa do tabuleiro de Scrabble. As operações básicas associadas a este TAD são:

- Construtor
 - $cria_casa: int \times int \mapsto casa$
 $cria_casa(lin, col)$ recebe dois inteiros correspondentes à linha lin e coluna col e devolve a casa correspondente. O construtor verifica a validade dos seus argumentos, gerando um `ValueError` com a mensagem '`cria_casa: argumentos inválidos`' caso os seus argumentos não sejam válidos.
- Seletores
 - $obtem_col: casa \mapsto int$
 $obtem_col(c)$ devolve a coluna col da casa c .
 - $obtem_lin: casa \mapsto int$
 $obtem_lin(c)$ devolve a linha lin da casa c .
- Reconhecedor
 - $eh_casa: universal \mapsto bool$
 $eh_casa(arg)$ devolve `True` caso o seu argumento seja um TAD *casa* e `False` caso contrário.
- Teste

³A representação interna dos elementos do tipo deve ser imutável e *hashable*.

- *casas_iguais*: $universal \times universal \mapsto bool$
casas_iguais(*c1*, *c2*) devolve **True** apenas se *c1* e *c2* são casas e são iguais, e **False** caso contrário.
- Transformador
 - *casa_para_str*: $casa \mapsto str$
casa_para_str(*c*) devolve a cadeia de caracteres que representa o seu argumento, como mostrado nos exemplos.
 - *str_para_casa*: $str \mapsto casa$
str_para_casa(*s*) devolve a casa representada pelo seu argumento.

As funções de alto nível associadas a este TAD são:

- *incrementa_casa*: $casa \times str \times int \mapsto casa$
incrementa_casa(*c*, *d*, *s*) devolve a casa dum tabuleiro de Scrabble a seguir de *c* na direção *d* e a distância *s* (inteiro positivo). Caso não exista, devolve a casa *c*.

Exemplos de interação:

```
>>> p1 = cria_casa(-1, 12)
Traceback (most recent call last): <...>
ValueError: cria_casa: argumentos inválidos
>>> c1 = cria_casa(1, 2)
>>> c2 = cria_casa(12, 15)
>>> casas_iguais(c1, c2)
False
>>> casa_para_str(c2)
'(12,15)'
>>> casas_iguais(c1, str_para_casa('(1,2)'))
True
>>> casa_para_str(incrementa_casa(c1, 'H', 2))
'(1,4)'
>>> casas_iguais(c2, incrementa_casa(c2, 'H', 1))
True
```

2.1.2 TAD *jogador* (1,5 valores)

O TAD *jogador* é usado para representar um jogador do jogo Scrabble, a sua pontuação e letras. Os jogadores podem ser *humanos* ou *agentes*. As operações básicas associadas a este TAD são:

- Construtor

- *cria_humano*: $str \mapsto jogador$
cria_humano(*nome*) recebe uma cadeia de caracteres (não vazia) a representar o nome do jogador e devolve o *jogador* de Scrabble humano com 0 pontos e sem letras. O construtor verifica a validade do seu argumento, gerando um erro com a mensagem '*cria_humano: argumento inválido*'.
- *cria_agente*: $: str \mapsto jogador$
cria_agente(*nível*) recebe uma cadeia de caracteres a representar o nível do jogador ('FACIL', 'MEDIO' ou 'DIFICIL') e devolve o *jogador* de Scrabble agente com 0 pontos e sem letras. O construtor verifica a validade do seu argumento, gerando um erro com a mensagem '*cria_agente: argumento inválido*'.
- Seletores
 - *jogador_identidade*: $jogador \mapsto str$
jogador_identidade(*j*) devolve o nome do jogador *j* se é um jogador *humano* ou o nível se é um jogador *agente*.
 - *jogador_pontos*: $jogador \mapsto int$
jogador_pontos(*j*) devolve os pontos do jogador *j*.
 - *jogador_letras*: $jogador \mapsto str$
jogador_letras(*j*) devolve a cadeia de caracteres ordenada com todas as letras do jogador *j*.
- Modificadores
 - *recebe_letra*: $jogador \times str \mapsto jogador$
recebe_letra(*j*, *l*) modifica destrutivamente o jogador *j* acrescentado a letra *l* às suas letras, e devolve o próprio *jogador*.
 - *usa_letra*: $jogador \times str \mapsto jogador$
usa_letra(*j*, *l*) modifica destrutivamente o jogador *j* retirando a letra *l* às suas letras, e devolve o próprio *jogador*.
 - *soma_pontos*: $jogador \times int \mapsto jogador$
soma_pontos(*j*, *p*) modifica destrutivamente o jogador *j* somando os pontos *p* à sua pontuação atual, e devolve o próprio *jogador*.
- Reconhecedor
 - *eh_jogador*: $universal \mapsto bool$
eh_jogador(*arg*) devolve **True** caso o seu argumento seja um TAD *jogador* e **False** caso contrário.
 - *eh_humano*: $universal \mapsto bool$
eh_humano(*arg*) devolve **True** caso o seu argumento seja um TAD *jogador* humano e **False** caso contrário.

- *eh_agente*: $universal \mapsto bool$
eh_agente(arg) devolve **True** caso o seu argumento seja um TAD *jogador* agente e **False** caso contrário.
- Teste
 - *jogadores_iguais*: $universal \times universal \mapsto bool$
jogadores_iguais(j1, j2) devolve **True** apenas se *j1* e *j2* forem *jogadores* e forem iguais.
- Transformador
 - *jogador_para_str*: $jogador \mapsto str$
jogador_para_str(t) devolve a cadeia de caracteres que representa o *jogador* como mostrado nos exemplos.

As funções de alto nível associadas a este TAD são:

- *distribui_letras*: $jogador \times list \times int \rightarrow jogador$
distribui_letras(jog, saco, num) retira um máximo de *num* letras do final da lista *saco* (potencialmente vazia) e as acrescenta ao jogador *jog*, devolvendo o jogador. A função modifica destrutivamente a lista de letras e o jogador passados como argumento.

Exemplos de interação:

```
>>> jog1 = cria_humano('Maria')
>>> jog2 = cria_agente('NIGHTMARE')
Traceback (most recent call last): <...>
ValueError: cria_agente: argumento inválido
>>> jog2 = cria_agente('FACIL')
>>> jogadores_iguais(jog1, jog2)
False
>>> _ = recebe_letra(jog1, 'A')
>>> letras = ['A', 'B', 'C', 'A', 'D']
>>> _ = distribui_letras(jog1, letras, 3)
>>> letras
['A', 'B']
>>> jogador_para_str(soma_pontos(jog1, 17))
'Maria ( 17): A A C D'
>>> jogador_para_str(distribui_letras(jog2, letras, 10))
'BOT(FACIL) ( 0): A B'
```

Tabela 1: Pontuação das letras no jogo de Scrabble.

letra	pontos	letra	pontos	letra	pontos	letra	pontos
A	1	F	4	M	1	S	1
B	3	G	4	N	3	T	1
C	2	H	4	O	1	U	1
Ç	3	I	1	P	2	V	4
D	2	J	5	Q	6	X	8
E	1	L	2	R	1	Z	8

2.1.3 TAD *vocabulario* (2,5 valores)

O TAD *vocabulario* é usado para representar o conjunto de palavras que podem ser utilizadas durante o jogo. Adicionalmente, o TAD *vocabulario* regista a pontuação das palavras utilizando a pontuação das letras mostrada na Tabela 1.

Recomenda-se que a estrutura de dados do TAD *vocabulario* seja organizada internamente por forma a facilitar a procura de palavras por comprimento e primeira letra.

As operações básicas associadas a este TAD são:

- Construtor
 - *cria_vocabulario*: $tuple \mapsto vocabulario$
cria_vocabulario(v) devolve o *vocabulario* que contém as palavras contidas no tuplo *v*. O construtor verifica a validade do seu argumento, gerando um erro com a mensagem '*cria_vocabulario: argumento inválido*'. Considere para este fim que o tuplo contém pelo menos uma palavra e que as palavras são cadeias de caracteres únicas de letras maiúsculas do abecedário Português de comprimento entre 2 e 15 letras.
- Seletores
 - *obtem_pontos*: $vocabulario \times str \mapsto int$
obtem_pontos(vocabulario, palavra) devolve os pontos da palavra do vocabulário, ou 0 caso não se encontre.
 - *obtem_palavras*: $vocabulario \times int \times str \mapsto tuple$
obtem_palavras(vocabulario, comp, letra) devolve um tuplo de pares que correspondem a todas as palavras com comprimento *comp* e primeira letra *letra*. Cada par do tuplo contém a palavra e a respetiva pontuação. Os pares devem estar ordenados por ordem decrescente de pontuação das palavras, e em caso de empate, em ordem lexicográfica. Caso não existam no vocabulário palavras com o comprimento e primeira letra indicados, a função deverá devolver um tuplo vazio.

- Teste

- *testa_palavra_padrao*: $\text{vocabulario} \times \text{str} \times \text{str} \times \text{str} \mapsto \text{bool}$
testa_palavra_padrao(*vocabulario*, *palavra*, *padrao*, *letras*) devolve **True** caso exista a palavra *palavra* no vocabulário *vocabulario* e seja possível formar a palavra fornecida substituindo os caracteres '.' do padrão *padrao* por letras de *letras*. Caso contrário, devolve **False**.

- Transformador

- *ficheiro_para_vocabulario*: $\text{str} \mapsto \text{vocabulario}$
ficheiro_para_vocabulario(*nome_fich*) devolve o vocabulário formado a partir das palavras contidas no ficheiro *nome_fich*. Considere que o ficheiro contém uma palavra por linha, podendo ter linhas vazias (que serão ignoradas). As palavras do ficheiro são sequências de caracteres únicas de comprimento arbitrário, contendo potencialmente qualquer carácter. Para a construção do vocabulário, considere as palavras de comprimento entre 2 e 15 letras do abecedário português, convertidas para letras maiúsculas.
- *vocabulario_para_str*: $\text{vocabulario} \mapsto \text{str}$
vocabulario_para_str(*vocabulario*) devolve uma cadeia de caracteres que concatena todas as palavras guardadas no vocabulário *vocabulario*, separadas por um carácter de mudança de linha. As palavras devem estar ordenadas por ordem crescente do seu comprimento e, para o mesmo comprimento, por ordem lexicográfica do seu primeiro carácter. Palavras que tenham o mesmo comprimento e o mesmo primeiro carácter, devem estar ordenadas pela ordem definida pelo seletor *obtem_palavras*. .

As funções de alto nível associadas a este TAD são:

- *procura_palavra_padrao*: $\text{vocabulario} \times \text{str} \times \text{str} \times \text{int} \mapsto \text{tuple}$
procura_palavra_padrao(*vocabulario*, *padrao*, *letras*, *min_pontos*) devolve o tuplo formado pela palavra e a pontuação, que correspondem à palavra do *vocabulario* com maior pontuação que é possível formar utilizando as letras da cadeia de caracteres *letras* para completar todos os espaços livres do padrão *padrao*, cumprindo a restrição de que a pontuação da palavra não poderá ser inferior a *min_pontos*. Caso a função não encontre nenhuma palavra, deverá devolver o tuplo ('', 0).

O mecanismo de procura da palavra deverá ser o seguinte:

- caso o padrão comece por uma letra, deverá aceder às palavras do vocabulário correspondentes ao comprimento e primeira letra da palavra, analisá-las, e devolver a palavra que emparelhe com o padrão, que tenha a melhor pontuação, com no mínimo uma pontuação de *min_pontos*; no caso de existirem várias palavras com essa pontuação deverá ser devolvida a primeira em ordem lexicográfica;

- caso o padrão comece por um espaço livre, deverá repetir o procedimento anterior, mas considerando que a letra inicial da palavra é cada uma das letras disponíveis no conjunto de letras, por ordem lexicográfica; no caso de existirem várias palavras com a melhor pontuação, deverá ser devolvida a primeira em ordem lexicográfica;

Exemplos de interação:

```
>>> vocab = cria_vocabulario(('asa', 'AULA', 'CAO', 'CASA', 'VACA'))
Traceback (most recent call last): <...>
ValueError: cria_vocabulario: argumento inválido
>>> tup = ('AULA', 'CAO', 'ASA', 'CEGO', 'CEGA', 'CONE', 'COÇA',
           'VACA', 'CASA')
>>> vocab = cria_vocabulario(tup)
>>> obtem_palavras(vocab, 4, 'C')
(('CEGA', 8), ('CEGO', 8), ('COÇA', 7), ('CONE', 7), ('CASA', 5))
>>> vocabulario_para_str(vocab)
'ASA\nCAO\nAULA\nCEGA\nCEGO\nCOÇA\nCONE\nCASA\nVACA'
>>> testa_palavra_padrao(vocab, 'CASA', 'C.SA', 'BCA')
True
>>> testa_palavra_padrao(vocab, 'BOLA', '.OLA', 'BCA')
False
>>> testa_palavra_padrao(vocab, 'CASA', 'CA.A', 'BCA')
False
>>> obtem_pontos(vocab, 'CASA')
5
>>> procura_palavra_padrao(vocab, '.A.A', 'CCVVA', 0)
('VACA', 8)
>>> procura_palavra_padrao(vocab, '.A.A', 'CCVVA', 8)
('VACA', 8)
>>> procura_palavra_padrao(vocab, '.A.A', 'CCVVA', 9)
('', 0)
>>> with open('vocab.txt', 'r') as f: print(f.read())
Asa
AuLa
cao
CAsa
INCONSTITUCIONALISSIMAMENTE
COME
vacA
>>> vocabulario_para_str(ficheiro_para_vocabulario('vocab.txt'))
'ASA\nCAO\nAULA\nCASA\nCOME\nVACA'
```

2.1.4 TAD *tabuleiro* (3,0 valores)

O TAD *tabuleiro* é usado para representar um tabuleiro do jogo Scrabble e as letras nele colocadas. As operações básicas associadas a este TAD são:

- Construtor
 - *cria_tabuleiro*: $\{\}$ $\mapsto$ *tabuleiro*
cria_tabuleiro() devolve um *tabuleiro* de Scrabble vazio.
- Seletores
 - *obtem_letra*: *tabuleiro* $\times$ *casa* $\mapsto$ *str*
obtem_letra(*t*, *c*) devolve a letra contida na casa *c* do tabuleiro *t*.
- Modificadores
 - *insere_letra*: *tabuleiro* $\times$ *casa* $\times$ *str* $\mapsto$ *tabuleiro*
insere_letra(*t*, *c*, *l*) modifica destrutivamente o tabuleiro *t* colocando a letra *l* na casa *c*, e devolve o próprio *tabuleiro*.
- Reconhecedor
 - *eh_tabuleiro*: *universal* $\mapsto$ *bool*
eh_tabuleiro(*arg*) devolve **True** caso o seu argumento seja um TAD *tabuleiro* e **False** caso contrário.
 - *eh_tabuleiro_vazio*: *universal* $\mapsto$ *bool*
eh_tabuleiro_vazio(*arg*) devolve **True** caso o seu argumento seja um TAD *tabuleiro* e estiver vazio (sem letras) e **False** caso contrário.
- Teste
 - *tabuleiros_iguais*: *universal* $\times$ *universal* $\mapsto$ *bool*
tabuleiros_iguais(*t1*, *t2*) devolve **True** apenas se *t1* e *t2* forem *tabuleiros* e forem iguais.
- Transformador
 - *tabuleiro_para_str*: *tabuleiro* $\mapsto$ *str*
tabuleiro_para_str(*t*) devolve a cadeia de caracteres que representa o *tabuleiro* como mostrado nos exemplos.

As funções de alto nível associadas a este TAD são:

- *obtem_padrao*: *tabuleiro* $\times$ *casa* $\times$ *casa* $\mapsto$ *str*
obtem_padrao(*t*, *i*, *f*) devolve a sequência de letras contida no tabuleiro *t* entre a casa *i* e a casa *f* (ambas inclusive) na mesma linha vertical ou horizontal.

- *insere_palavra*: $tabuleiro \times casa \times str \times str \mapsto tabuleiro$
insere_palavra(t, c, d, p) modifica destrutivamente o tabuleiro *t* colocando a palavra *p* na casa *c* na direção *d*, e devolve o próprio *tabuleiro*.
- *obtem_subpadroes*: $tabuleiro \times casa \times casa \times int \mapsto tuple \times tuple$
obtem_subpadroes(t, i, f, l) devolve dois tuplos de igual tamanho. O primeiro tuplo contém todos os subpadrões viáveis ordenados gerados a partir do padrão original contido no tabuleiro *t* entre as casas *i* e *f* (ambas inclusive) como descrito na Secção 1.2, com no máximo *l* espaços livres. O segundo tuplo contém a casa onde começa cada um dos o subpadrões correspondentes do primeiro tuplo. As casas *i* e *f* pertencem à mesma linha vertical ou horizontal.
- *gera_todos_padroes*: $tabuleiro \times int \mapsto tuple \times tuple \times tuple$
gera_todos_padroes(t, l) devolve três tuplos de igual tamanho. O primeiro contém todos os sub-padrões viáveis do tabuleiro que contenham no máximo *l* espaços livres, formado pelos sub-padrões ordenados obtidos de cada uma das linhas completas do tabuleiro (da primeira à última), seguidos dos sub-padrões ordenados obtidos de cada coluna completa (da primeira até a última). O segundo e terceiro tuplos, correspondem à casa de início e à direção ('V' ou 'H') do sub-padrão correspondente do primeiro tuplo.

Exemplos de interação:

```
>>> tab = cria_tabuleiro()
>>> c1, c2 = cria_casa(8,6), cria_casa(6,8)
>>> tab = insere_letra(tab, c1, 'A')
>>> tab = insere_letra(tab, c2, 'B')
>>> obtem_letra(tab, c1), obtem_letra(tab, c2)
('A', 'B')
>>> tab = cria_tabuleiro()
>>> _ = insere_palavra(tab, cria_casa(8,3), 'H', 'COMPUTADOR')
>>> obtem_padrao(tab, cria_casa(8,10), cria_casa(8,12))
'DOR'
>>> obtem_padrao(tab, cria_casa(7,5), cria_casa(10,5))
'.M..'
>>> tab = cria_tabuleiro()
>>> _ = insere_palavra(tab, cria_casa(8,1), 'H', 'FUNDAMENTOS')
>>> _ = insere_palavra(tab, cria_casa(8,4), 'V', 'DA')
>>> _ = insere_palavra(tab, cria_casa(2,6), 'V', 'PROGRAMAÇÃO')
>>> print(tabuleiro_para_str(tab))

          1 1 1 1 1 1
    1 2 3 4 5 6 7 8 9 0 1 2 3 4 5
+-----+
1 | . . . . . . . . . . . . . . |
```

```

2 | . . . . . P . . . . . |
3 | . . . . . R . . . . . |
4 | . . . . . O . . . . . |
5 | . . . . . G . . . . . |
6 | . . . . . R . . . . . |
7 | . . . . . A . . . . . |
8 | F U N D A M E N T O S . . . . |
9 | . . . A . A . . . . . |
10 | . . . . . Ç . . . . . |
11 | . . . . . A . . . . . |
12 | . . . . . O . . . . . |
13 | . . . . . . . . . . . |
14 | . . . . . . . . . . . |
15 | . . . . . . . . . . . |
  +-----+
>>> pat,cas = obtem_subpadroes(tab, cria_casa(1,6), cria_casa(15,6), 4)
>>> pat
('.PROGRAMAÇÃO...', '.PROGRAMAÇÃO..', '.PROGRAMAÇÃO.', '.PROGRAMAÇÃO',
 'PROGRAMAÇÃO...', 'PROGRAMAÇÃO..', 'PROGRAMAÇÃO.')
>>> tuple(casa_para_str(c) for c in cas)
('(1,6)', '(1,6)', '(1,6)', '(1,6)', '(2,6)', '(2,6)', '(2,6)')
>>> pat,cas = obtem_subpadroes(tab, cria_casa(1,6), cria_casa(15,6), 3)
>>> pat
(('PROGRAMAÇÃO..', '.PROGRAMAÇÃO.', '.PROGRAMAÇÃO', 'PROGRAMAÇÃO...',
 'PROGRAMAÇÃO..', 'PROGRAMAÇÃO.'),
>>> pat, cas, dir = gera_todos_padroes(tab,7)
>>> len(pat), len(cas), len(dir)
(680, 680, 680)
>>> pat[:100]
('.....P..', '.....G..', '...A', '...O...', '.....N...',
 '.PROGRAMAÇÃO.', '.T..')
>>> tuple(casa_para_str(c) for c in cas[:100])
('(2,1)', '(5,2)', '(9,1)', '(12,3)', '(4,3)', '(1,6)', '(7,9)')
>>> dir[:100]
('H', 'H', 'H', 'H', 'V', 'V', 'V')

```

2.2 Funções adicionais

2.2.1 baralha_saco: $int \rightarrow list$ (0,5 valores)

baralha_saco(seed) é uma função auxiliar que recebe um inteiro positivo *seed* representando o estado inicial do gerador pseudo-aleatório da Secção 2.2 do primeiro projeto, e devolve uma lista *baralhada* com todas as letras contidas no saco de Scrabble. Como no primeiro projeto, para baralhar as letras a sua função deverá: 1) construir uma lista

com todas as letras contidas no saco em ordem lexicográfico; e 2) permutar as letras. A Tabela 2 mostra o número de ocorrências das letras no saco de Scrabble.

Tabela 2: Número de ocorrências das letra no saco do jogo de Scrabble.

letra	#	letra	#	letra	#	letra	#
A	14	F	2	M	6	S	8
B	3	G	2	N	4	T	5
C	4	H	2	O	10	U	7
Ç	2	I	10	P	4	V	2
D	5	J	2	Q	1	X	1
E	11	L	5	R	6	Z	1

```
>>> baralha_saco(9)[:10]
['S', 'N', 'A', 'J', 'U', 'T', 'S', 'U', 'M', 'C']
>>> baralha_saco(10)[:10]
['T', 'O', 'L', 'I', 'O', 'C', 'G', 'H', 'S', 'O']
```

2.2.2 jogada_humano: $tabuleiro \times jogador \times vocabulario \times list \mapsto bool$ (1,0 valores)

jogada_humano(*tab*, *jog*, *vocab*, *pilha*) é uma função auxiliar que recebe um *tabuleiro*, um *jogador* humano, um *vocabulario* e uma lista de letras. A função processa o turno completo do jogador humano devendo apresentar a mensagem do exemplo a seguir, repetindo a mensagem até o jogador introduzir uma jogada válida num formato dependente da ação pretendida:

- **Passar:** 'P'. Neste caso, a função devolve **False** sem alterar nenhum dos argumentos.
- **Trocar:** 'T <seq_letras>', sendo <seq_letras> a sequência de uma ou mais letras separada por espaços do conjunto de letras do jogador para trocar. Caso a jogada seja válida, a função devolve **True**, modifica o jogador retirando novas letras do final da lista de letras (que é também modificada).
- **Jogar:** 'J <linha> <coluna> <dir> <palavra>'. Caso a jogada seja válida, a função devolve **True** e modifica o tabuleiro, atualiza o jogador, atualizando a sua pontuação e retirando novas letras do final da lista de letras (que é também modificada). No caso da primeira jogada da partida (o tabuleiro não contém nenhuma palavra), a palavra colocada deve cobrir a casa central do tabuleiro. Para as jogadas seguintes serem válidas, as casas do tabuleiro onde se colocarem as palavras devem ser um padrão viável, e as palavras devem estar no *vocabulario*.

```

>>> tab = cria_tabuleiro()
>>> jog1 = cria_humano('Maria')
>>> for let in 'AAUOTXF': _ = recebe_letra(jog1, let)
>>> pilha = ['S', 'B', 'P', 'E', 'C', 'E', 'E', 'S', 'J', 'D', 'I']
>>> vocab = cria_vocabulario(('TOFU', 'LUTA', 'USA'))
>>> jogada_humano(tab, jog1, vocab, pilha)
Jogada Maria: olà
Jogada Maria: P
False
>>> jogada_humano(tab, jog1, vocab, pilha)
Jogada Maria: T X A
True
>>> jogador_para_str(jog1)
'Maria ( 0): A D F I O T U'
>>> pilha
['S', 'B', 'P', 'E', 'C', 'E', 'E', 'S', 'J']
>>> jogada_humano(tab, jog1, vocab, pilha)
Jogada Maria: J 7 8 V LUTA
Jogada Maria: J 7 8 V TOFU
True
>>> jogador_para_str(jog1)
'Maria ( 7): A D E E I J S'
>>> jogada_humano(tab, jog1, vocab, pilha)
Jogada Maria: J 10 6 H SEU
Jogada Maria: J 10 8 H USA
True
>>> jogador_para_str(jog1)
'Maria ( 10): C D E E E I J'
>>> pilha
['S', 'B', 'P']
>>> print(tabuleiro_para_str(tab))

```

2.2.3 jogada_agente: *tabuleiro* × *jogador* × *vocabulario* × *list* ⇨ *bool* (1,5 valores)

jogada_agente(*tab*, *jog*, *vocab*, *pilha*) é uma função auxiliar que recebe um *tabuleiro*, um *jogador* agente, um *vocabulario*⁴ e uma lista de letras, e realiza uma das seguintes ações:

- **Passar**, se for a primeira jogada (tabuleiro vazio) ou, caso contrário, se não conseguir nem Jogar nem Trocar (ver a seguir). Neste caso, a função devolve **False** sem alterar nenhum dos argumentos.

⁴Ficheiro de vocabulário dos exemplos desta secção: <https://fenix.tecnico.ulisboa.pt/downloadFile/1126518382353939/vocab25k.txt>

- **Trocar**, caso não consiga Jogar (ver a seguir) e existam pelo menos sete letras no saco, troca todas as letras. Neste caso, a função devolve **True**, modifica o jogador retirando novas letras do final da lista de letras (que é também modificada).
- **Jogar**, devolve **True** e modifica o tabuleiro, atualiza o jogador, atualizando a sua pontuação e retirando novas letras do final da lista de letras (que é também modificada). Para escolher a palavra, a casa e a direção a função deve:
 1. gerar todos os padrões possíveis para a configuração atual do tabuleiro, através da função *gera_todos_padroes*, tendo em conta o número de letras do jogador;
 2. selecionar um de cada N padrões (operação *slicing* [::N]) dos obtidos anteriormente, sendo $N = 100$ se *nivel* for 'FACIL', $N = 50$ se *nivel* for 'MEDIO' e $N = 10$ se *nivel* for 'DIFICIL';
 3. sobre cada um dos padrões resultantes, invocar a função *procura_palavra_padrao* e selecionar a primeira palavra obtida com a maior pontuação.

A função apresenta a mensagem da jogada realizada imitando o formato utilizado pelos jogadores humanos como mostrado nos exemplos:

```
>>> tab = cria_tabuleiro()
>>> pilha = ['N', 'R', 'R', 'S', 'S', 'B', 'A', 'L', 'V', 'M', 'D',
            'A', 'E', 'R', 'E', 'M', 'I', 'O', 'G', 'O']
>>> bot = cria_agente('FACIL')
>>> vocab = ficheiro_para_vocabulario('vocab25k.txt')
>>> _ = distribui_letras(bot, pilha, 7)
>>> jogada_agente(tab, bot, vocab, pilha)
Jogada FACIL: P
False
>>> _ = insere_palavra(tab, cria_casa(8,1), 'H', 'FUNDAMENTOS')
>>> _ = insere_palavra(tab, cria_casa(8,4), 'V', 'DA')
>>> _ = insere_palavra(tab, cria_casa(2,6), 'V', 'PROGRAMAÇÃO')
>>> jogada_agente(tab, bot, vocab, pilha)
Jogada FACIL: J 9 1 H GERA
True
>>> jogada_agente(tab, bot, vocab, pilha)
Jogada FACIL: T A D E I M O O
True
>>> jogada_agente(tab, bot, vocab, pilha)
Jogada FACIL: P
False
>>> jogador_para_str(bot)
'BOT(FACIL) ( 7): A B L M S S V'
>>> pilha
['N', 'R', 'R']
```

```

>>> tab = cria_tabuleiro()
>>> pilha = ['N', 'R', 'R', 'S', 'S', 'B', 'A', 'L', 'V', 'M', 'D',
            'A', 'E', 'R', 'E', 'M', 'I', 'O', 'G', 'O']
>>> bot = cria_agente('MEDIO')
>>> vocab = ficheiro_para_vocabulario('vocab25k.txt')
>>> _ = distribui_letras(bot, pilha, 7)
>>> _ = insere_palavra(tab, cria_casa(8,1), 'H', 'FUNDAMENTOS')
>>> _ = insere_palavra(tab, cria_casa(8,4), 'V', 'DA')
>>> _ = insere_palavra(tab, cria_casa(2,6), 'V', 'PROGRAMAÇÃO')
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada MEDIO: J 9 1 H GERA
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada MEDIO: J 8 8 V NODOA
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada MEDIO: J 5 4 V AVIDA
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada MEDIO: J 5 1 H BELA
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada MEDIO: J 12 5 H MORA
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada MEDIO: P
>>> jogador_para_str(bot)
'BOT(MEDIO) ( 35): M N R S S'
>>> pilha
[]

```

```

>>> tab = cria_tabuleiro()
>>> pilha = ['N', 'R', 'R', 'S', 'S', 'B', 'A', 'L', 'V', 'M', 'D',
            'A', 'E', 'R', 'E', 'M', 'I', 'O', 'G', 'O']
>>> bot = cria_agente('DIFICIL')
>>> vocab = ficheiro_para_vocabulario('vocab25k.txt')
>>> _ = distribui_letras(bot, pilha, 7)
>>> _ = insere_palavra(tab, cria_casa(8,1), 'H', 'FUNDAMENTOS')
>>> _ = insere_palavra(tab, cria_casa(8,4), 'V', 'DA')
>>> _ = insere_palavra(tab, cria_casa(2,6), 'V', 'PROGRAMAÇÃO')
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada DIFICIL: J 6 3 V IGNOREM
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada DIFICIL: J 5 4 H LEGADO
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada DIFICIL: J 4 2 H BRAVO
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada DIFICIL: J 7 10 V SONS

```



```

>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada DIFICIL: J 6 2 H MIR
>>> _ = jogada_agente(tab, bot, vocab, pilha)
Jogada DIFICIL: P
>>> jogador_para_str(bot)
'BOT(DIFICIL) ( 42):'
>>> pilha
[]
>>> print(tabuleiro_para_str(tab))
               1 1 1 1 1 1
            1 2 3 4 5 6 7 8 9 0 1 2 3 4 5
      +-----+
1  | . . . . . . . . . . . . . . . |
2  | . . . . . P . . . . . . . . . |
3  | . . . . . R . . . . . . . . . |
4  | . B R A V O . . . . . . . . . |
5  | . . . L E G A D O . . . . . . |
6  | . M I R . R . . . . . . . . . |
7  | . . G . . A . . . S . . . . . |
8  | F U N D A M E N T O S . . . . . |
9  | . . O A . A . . . N . . . . . |
10 | . . R . . Ç . . . S . . . . . |
11 | . . E . . A . . . . . . . . . |
12 | . . M . . O . . . . . . . . . |
13 | . . . . . . . . . . . . . . . |
14 | . . . . . . . . . . . . . . . |
15 | . . . . . . . . . . . . . . . |
      +-----+

```

2.2.4 scrabble2: *tuple* × *str* × *int* ⇨ tuplo (1,0 valores)

scrabble2(jogadores, nome_fich, seed) é a função principal que permite jogar um jogo completo de Scrabble2 de dois a quatro jogadores. A função recebe um tuplo com o nome dos jogadores humanos (cadeia de caracteres não vazia) e o nível dos jogadores agentes (cadeia de caracteres a começar por '@' seguido do nível) na ordem em que jogam, um nome de ficheiro com o vocabulário e um inteiro positivo representando o estado inicial do gerador pseudo-aleatório, e devolve o tuplo com a pontuação final obtida pelos jogadores. O jogo começa baralhando o saco de letras e distribuindo o conjunto de 7 letras a cada um dos jogadores em ordem. O jogo desenrola-se depois conforme as regras e como mostrado no exemplo a seguir. O jogo termina quando todos os jogadores passam ou quando um jogador fica sem letras e o saco estiver esgotado. A função deve verificar a validade dos seus argumentos, gerando um erro com a mensagem 'scrabble2: argumentos inválidos'.

Exemplo 1 O seguinte exemplo mostra as primeiras jogadas de um jogo de Scrabble2 entre um jogador humano e dois agentes. A interação completa encontra-se no repositório dos testes públicos⁵.

```
>>> jog = ('Maria', '@MEDIO', '@DIFICIL', )
>>> scrabble2(jog, 'vocab25k.txt', 32)
Bem-vindo ao SCRABBLE2.
```

```

              1 1 1 1 1 1
            1 2 3 4 5 6 7 8 9 0 1 2 3 4 5
      +-----+
1 | . . . . . |
2 | . . . . . |
3 | . . . . . |
4 | . . . . . |
5 | . . . . . |
6 | . . . . . |
7 | . . . . . |
8 | . . . . . |
9 | . . . . . |
10 | . . . . . |
11 | . . . . . |
12 | . . . . . |
13 | . . . . . |
14 | . . . . . |
15 | . . . . . |
      +-----+
```

```
Maria ( 0): B Ç D E M O U
BOT(MEDIO) ( 0): A B I N O O V
BOT(DIFICIL) ( 0): A A B C E I T
Jogada Maria: J 8 6 H MU DO
```

```

              1 1 1 1 1 1
            1 2 3 4 5 6 7 8 9 0 1 2 3 4 5
      +-----+
1 | . . . . . |
2 | . . . . . |
3 | . . . . . |
4 | . . . . . |
5 | . . . . . |
6 | . . . . . |
7 | . . . . . |
8 | . . . . M U D O . . . . . |
9 | . . . . . |
      +-----+
```

⁵<https://gitlab.rnl.tecnico.ulisboa.pt/ist-fp/fp25/prj2/tests/public>

```

10 | . . . . . |
11 | . . . . . |
12 | . . . . . |
13 | . . . . . |
14 | . . . . . |
15 | . . . . . |
+-----+

```

```

Maria ( 5): A B Ç E R S U
BOT(MEDIO) ( 0): A B I N O O V
BOT(DIFICIL) ( 0): A A B C E I T
Jogada MEDIO: J 5 9 V NOVO

```

```

               1 1 1 1 1 1
            1 2 3 4 5 6 7 8 9 0 1 2 3 4 5
+-----+
1 | . . . . . |
2 | . . . . . |
3 | . . . . . |
4 | . . . . . |
5 | . . . . . N . . . . . |
6 | . . . . . O . . . . . |
7 | . . . . . V . . . . . |
8 | . . . . . M U D O . . . . . |
9 | . . . . . |
10 | . . . . . |
11 | . . . . . |
12 | . . . . . |
13 | . . . . . |
14 | . . . . . |
15 | . . . . . |
+-----+

```

```

Maria ( 5): A B Ç E R S U
BOT(MEDIO) ( 9): A A B E I O O
BOT(DIFICIL) ( 0): A A B C E I T
Jogada DIFICIL: J 6 8 V ABDICA

```

```

               1 1 1 1 1 1
            1 2 3 4 5 6 7 8 9 0 1 2 3 4 5
+-----+
1 | . . . . . |
2 | . . . . . |
3 | . . . . . |
4 | . . . . . |
5 | . . . . . N . . . . . |
6 | . . . . . A O . . . . . |

```

```

7 | . . . . . B V . . . . . |
8 | . . . . . M U D O . . . . . |
9 | . . . . . I . . . . . |
10 | . . . . . C . . . . . |
11 | . . . . . A . . . . . |
12 | . . . . . . . . . . . |
13 | . . . . . . . . . . . |
14 | . . . . . . . . . . . |
15 | . . . . . . . . . . . |

```

```

+-----+

```

```

Maria ( 5): A B Ç E R S U
BOT(MEDIO) ( 9): A A B E I O O
BOT(DIFICIL) ( 10): A E M O O O T
Jogada Maria: J 3 9 V RENOVOU

```

```

1 1 1 1 1 1
1 2 3 4 5 6 7 8 9 0 1 2 3 4 5

```

```

+-----+

```

```

1 | . . . . . . . . . . . |
2 | . . . . . . . . . . . |
3 | . . . . . . R . . . . . |
4 | . . . . . E . . . . . |
5 | . . . . . N . . . . . |
6 | . . . . . A O . . . . . |
7 | . . . . . B V . . . . . |
8 | . . . . . M U D O . . . . . |
9 | . . . . . I U . . . . . |
10 | . . . . . C . . . . . |
11 | . . . . . A . . . . . |
12 | . . . . . . . . . . . |
13 | . . . . . . . . . . . |
14 | . . . . . . . . . . . |
15 | . . . . . . . . . . . |

```

```

+-----+

```

```

Maria ( 17): A B C Ç E I S
BOT(MEDIO) ( 9): A A B E I O O
BOT(DIFICIL) ( 10): A E M O O O T
Jogada MEDIO: J 5 9 H NBA

```

```

1 1 1 1 1 1
1 2 3 4 5 6 7 8 9 0 1 2 3 4 5

```

```

+-----+

```

```

1 | . . . . . . . . . . . |
2 | . . . . . . . . . . . |
3 | . . . . . R . . . . . |

```

```

4 | . . . . . E . . . . . |
5 | . . . . . N B A . . . . |
6 | . . . . . A O . . . . . |
7 | . . . . . B V . . . . . |
8 | . . . . . M U D O . . . . |
9 | . . . . . I U . . . . . |
10 | . . . . . C . . . . . |
11 | . . . . . A . . . . . |
12 | . . . . . . . . . . . |
13 | . . . . . . . . . . . |
14 | . . . . . . . . . . . |
15 | . . . . . . . . . . . |

```

-----+

Maria (17): A B C Ç E I S
BOT(MEDIO) (16): A E E I O O S
BOT(DIFICIL) (10): A E M O O O T
Jogada DIFICIL: J 10 8 H COMETA

1 1 1 1 1 1
1 2 3 4 5 6 7 8 9 0 1 2 3 4 5

```

1 | . . . . . . . . . . . |
2 | . . . . . . . . . . . |
3 | . . . . . R . . . . . |
4 | . . . . . E . . . . . |
5 | . . . . . N B A . . . . |
6 | . . . . . A O . . . . . |
7 | . . . . . B V . . . . . |
8 | . . . . . M U D O . . . . |
9 | . . . . . I U . . . . . |
10 | . . . . . C O M E T A . . |
11 | . . . . . A . . . . . |
12 | . . . . . . . . . . . |
13 | . . . . . . . . . . . |
14 | . . . . . . . . . . . |
15 | . . . . . . . . . . . |

```

-----+

Maria (17): A B C Ç E I S
BOT(MEDIO) (16): A E E I O O S
BOT(DIFICIL) (17): F G H O O O T
Jogada Maria:

3 Condições de Realização e Prazos

- A entrega do 1º projeto será efetuada exclusivamente por via eletrónica. Para submeter o seu projeto deverá realizar pelo menos uma atualização do repositório remoto GitLab fornecido pelo corpo docente, até às **17:00 do dia 24 de Outubro de 2025**. Depois desta hora, qualquer atualização do repositório será ignorada. Não serão aceites submissões de projetos por outras vias sob pretexto algum.
- A solução do projeto deverá consistir apenas num único ficheiro com extensão *.py* contendo todo o código do seu projeto. Os alunos não devem alterar a estrutura do seu repositório, nem acrescentar novos ficheiros, apenas precisam editar o ficheiro com extensão *.py*.
- Cada aluno tem direito a **15 submissões sem penalização**. Por cada submissão adicional serão descontados 0,1 valores na componente de avaliação automática.
- Será considerada para avaliação a **última** submissão (mesmo que tenha pontuação inferior a submissões anteriores). Deverá, portanto, verificar cuidadosamente que a última entrega realizada corresponde à versão do projeto que pretende que seja avaliada.
- Submissões que não corram nenhum dos testes automáticos por causa de pequenos erros de sintaxe ou de codificação, poderão ser corrigidos pelo corpo docente, incorrendo numa penalização de três valores.
- Não é permitida a utilização de qualquer módulo ou função não disponível *built-in* no Python 3, ou seja, não são permitidos `import`, com exceção da função `reduce` do `functools`.
- Pode, ou não, haver uma discussão oral do trabalho e/ou uma demonstração do funcionamento do programa (será decidido caso a caso).
- Lembre-se de que no Técnico, a fraude académica é levada muito a sério e que a cópia em qualquer momento de avaliação (projetos incluídos) leva à reprovação na disciplina e eventualmente a um processo disciplinar. Os projetos serão submetidos a um sistema automático de deteção de cópias⁶, o corpo docente da cadeira será o único juiz do que se considera ou não copiar num projeto.
- A submissão do projeto por parte dos alunos, é interpretada pelo corpo docente como uma declaração de honra conforme cada aluno (ou grupo) é o autor único de todo o trabalho apresentado.

4 Submissão

A submissão do projeto de FP é realizada atualizando o repositório remoto GitLab privado fornecido pelo corpo docente a cada aluno (ou grupo de projeto). O endereço

⁶<https://theory.stanford.edu/~aiken/moss>

web do repositório do projeto dos alunos é [https://gitlab.rnl.tecnico.ulisboa.pt/ist-fp/fp25/prj2/\(curso\)/\(grupo\)](https://gitlab.rnl.tecnico.ulisboa.pt/ist-fp/fp25/prj2/(curso)/(grupo)), onde:

- (curso) pode ser leic-a, leic-t, leti ou leme;
- (grupo) pode ser:
 - o ist-id para os alunos da LEIC-A, LEIC-T e LETI (ex. ist190000);
 - g seguido dos dois dígitos que identificam o número de grupo de projeto dos alunos da LEME (ex. g00).

Sempre que é realizada uma nova atualização do repositório remoto é desencadeado o processo de avaliação automática do projeto e é contabilizada uma nova submissão. Quando a submissão tiver sido processada, poderá visualizar um relatório de execução com os detalhes da avaliação automática do seu projeto em [http://fp.rnl.tecnico.ulisboa.pt/fp/fp25p2/reports/\(grupo\)/](http://fp.rnl.tecnico.ulisboa.pt/fp/fp25p2/reports/(grupo)/). Adicionalmente, receberá no seu email o mesmo relatório. Se não receber o email ou o relatório web aparentar não ter sido atualizado, contacte com o corpo docente. Note que o sistema de submissão e avaliação não limita o número de submissões simultâneas. Um número elevado de submissões num determinado momento, poderá ocasionar a rejeição de alguns pedidos de avaliação. Para evitar problemas de último momento, **recomenda-se que submeta o seu projeto atempadamente**.

Detalhes sobre como aceder ao GitLab, configurar o par de chaves SSH, executar os comandos de Git e recomendações sobre ferramentas, encontram-se na página da disciplina na secção “Material de Apoio - Ambiente de Desenvolvimento”⁷.

5 Classificação

A nota do projeto será baseada nos seguintes aspetos:

1. **Execução correta (60%).** A avaliação da correta execução será feita com um conjunto de testes unitários utilizando o módulo de Python pytest⁸. Serão usados um conjunto de testes públicos (disponibilizados na página da disciplina) e um conjunto de testes privados. Como os testes de execução valem 60% (equivalente a 12 valores) da nota do projeto, uma submissão obtém a nota máxima de 1200 pontos por esta componente. O facto de um projeto completar com sucesso os testes públicos fornecidos não implica que esse projeto esteja totalmente correto, pois estes não são exaustivos. É da responsabilidade de cada aluno garantir que o código produzido está de acordo com a especificação do enunciado usando testes próprios adicionais, de forma a completar com sucesso os testes privados.

⁷<https://fenix.tecnico.ulisboa.pt/disciplinas/FProg3112/2025-2026/1-semester/ambiente-de-desenvolvimento>

⁸<https://docs.pytest.org/en/7.4.x/>

2. **Respeito pelas barreiras de abstração (20%).** A avaliação do respeito pelas barreiras de abstração também será feita automaticamente utilizando o módulo de Python `pytest`. Para este fim, serão usados um conjunto de testes (diferentes dos testes de execução) especificamente definidos para testar que o código desenvolvido pelos alunos respeita as barreiras de abstração. Como os testes de abstração valem 20% (equivalente a 4 valores) da nota do projeto, uma submissão obtém a nota máxima de 400 pontos por esta componente.
3. **Avaliação manual (20%).** Estilo de programação e facilidade de leitura. Em particular, serão consideradas as seguintes componentes:
 - Boas práticas (1,5 valores): serão considerados entre outros a clareza do código, elementos de programação funcional, integração de conhecimento adquirido durante a UC, a criatividade das soluções propostas e a escolha da representação adotada nos TADs.
 - Comentários (1 valor): deverão incluir a assinatura dos TADs (incluindo representação interna adotada e assinatura das operações básicas), assim como a assinatura de cada função definida, comentários para o utilizador (*docstring*) e comentários para o programador.
 - Tamanho de funções, duplicação de código e abstração procedimental (1 valor).
 - Escolha de nomes (0,5 valores).

6 Recomendações e aspetos a evitar

As seguintes recomendações e aspetos correspondem a sugestões para evitar maus hábitos de trabalho (e, conseqüentemente, más notas no projeto):

- Leia todo o enunciado, procurando perceber o objetivo das várias funções pedidas. Em caso de dúvida de interpretação, utilize o horário de dúvidas para esclarecer as suas questões.
- No processo de desenvolvimento do projeto, comece por implementar as várias funções pela ordem apresentada no enunciado, seguindo as metodologias estudadas na disciplina.
- Para verificar a funcionalidade das suas funções, utilize os exemplos fornecidos como casos de teste. Tenha o cuidado de reproduzir fielmente as mensagens de erro e restantes *outputs*, conforme ilustrado nos vários exemplos.
- Não pense que o projeto se pode fazer nos últimos dias. Se apenas iniciar o seu trabalho neste período irá sentir a Lei de Murphy em funcionamento (todos os problemas são mais difíceis do que parecem; tudo demora mais tempo do que nós pensamos; e se alguma coisa puder correr mal, ela vai correr mal, na pior das alturas possíveis).

- Não duplique código. Se duas funções são muito semelhantes é natural que estas possam ser fundidas numa única, eventualmente com mais argumentos.
- Não se esqueça que as funções excessivamente grandes são penalizadas no que respeita ao estilo de programação.
- A atitude “vou pôr agora o programa a correr de qualquer maneira e depois preocupo-me com o estilo” é totalmente errada.
- Quando o programa gerar um erro, preocupe-se em descobrir qual a causa do erro. As “marteladas” no código têm o efeito de distorcer cada vez mais o código.